

TASK 16 :ASSIGNMENT NO 16

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repositories link: <https://github.com/anujaw193-star/python-project-collection>

Q1. Load the insurance dataset using pandas and display the first 10 rows of the the data.

CODE:

```
[3]
✓ 0s # Load the dataset
df = pd.read_csv('./insurance - insurance.csv')
# Display the first 10 rows
print(df.head(10))
```

Output:

✓	...	age	sex	bmi	children	smoker	region	expenses
	0	19	female	27.9	0	yes	southwest	16884.92
	1	18	male	33.8	1	no	southeast	1725.55
	2	28	male	33.0	3	no	southeast	4449.46
	3	33	male	22.7	0	no	northwest	21984.47
	4	32	male	28.9	0	no	northwest	3866.86
	5	31	female	25.7	0	no	southeast	3756.62
	6	46	female	33.4	1	no	southeast	8240.59
	7	37	female	27.7	3	no	northwest	7281.51
	8	37	male	29.8	2	no	northeast	6406.41
	9	60	female	25.8	0	no	northwest	28923.14

Q2. Use pandas functions to get basic information about the dataset:

Shape of the data (shape)

Column names and data types (info())

Statistical summary of numerical columns (describe())

Code:

```
[5]
✓ 0s # 1. Shape of the dataset
print("Shape of the dataset:")
print(df.shape)
# 2. Column names and data types
print("\nDataset Information:")
df.info()
# 3. Statistical summary of numerical columns
print("\nStatistical Summary:")
print(df.describe())
```

Output:

```
✓ ... Shape of the dataset:
(1338, 7)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   age         1338 non-null   int64
1   sex         1338 non-null   object
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   object
5   region      1338 non-null   object
6   expenses    1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB

Statistical Summary:
```

	age	bmi	children	expenses
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.665471	1.094918	13270.422414
std	14.049960	6.098382	1.205493	12110.011240
min	18.000000	16.000000	0.000000	1121.870000
25%	27.000000	26.300000	0.000000	4740.287500
50%	39.000000	30.400000	1.000000	9382.030000
75%	51.000000	34.700000	2.000000	16639.915000
max	64.000000	53.100000	5.000000	63770.430000

Q3. Check if the dataset contains any null/missing values. Show the count of null values for each column.

Code :

```
[6]
✓ 0s # Check for null values in each column
print("Null values in each column:")
print(df.isnull().sum())
```

Output:

```
▼ ... Null values in each column:  
age      0  
sex      0  
bmi      0  
children 0  
smoker   0  
region   0  
expenses 0  
dtype: int64
```

Q4. Identify numerical and categorical columns in the dataset. Print them separately.

Code :

```
[17] ✓ Os # Identify numerical columns  
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns  
  
# Identify categorical columns  
categorical_cols = df.select_dtypes(include=['object', 'category']).columns  
# Print them separately  
print("Numerical Columns:")  
print(numerical_cols)  
  
print("\nCategorical Columns:")  
print(categorical_cols)
```

Output:

```
▼ ... Numerical Columns:  
Index(['age', 'bmi', 'children', 'expenses'], dtype='object')  
  
Categorical Columns:  
Index(['sex', 'smoker', 'region'], dtype='object')
```

Q5. Create distribution plots for all numerical columns (age,bmi,children,charges):

Use sns.histplot() or sns.distplot()

Create one plot per column (total 4 plots)

Add meaningful titles and labels

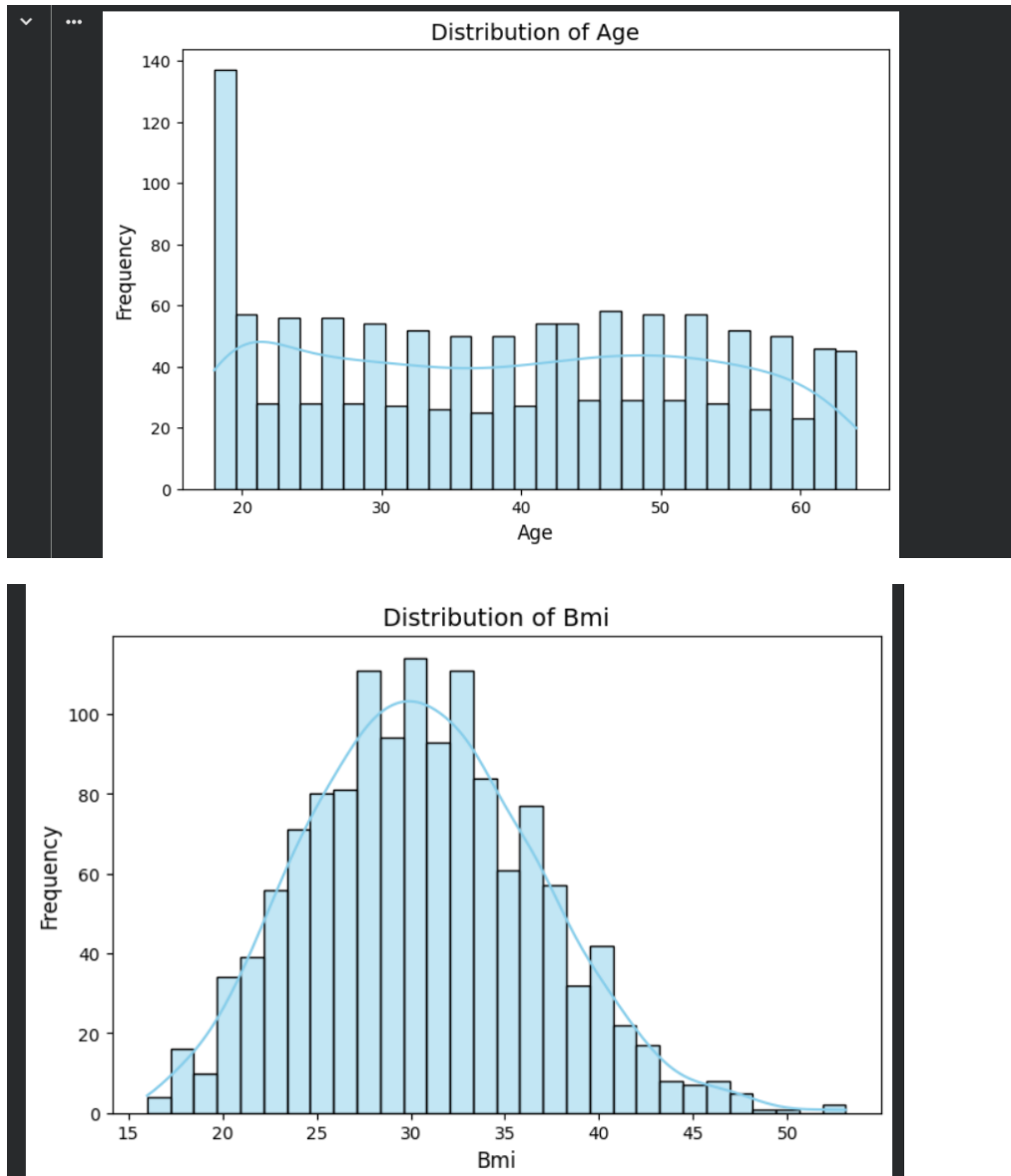
Code:

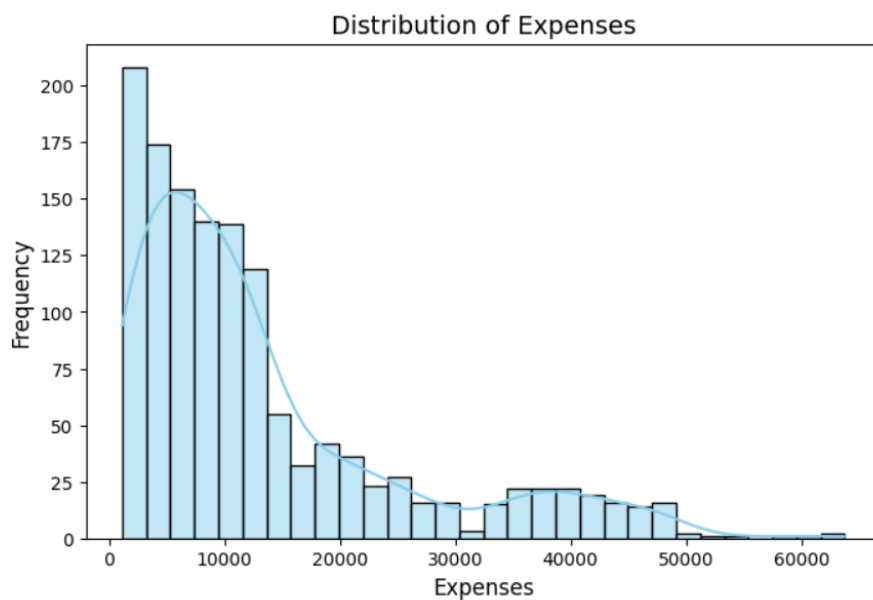
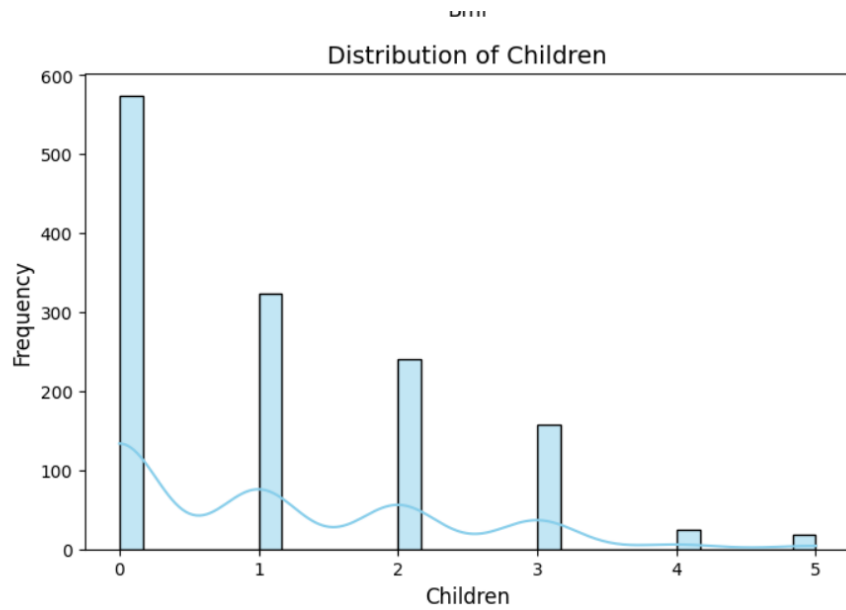
```

# Load dataset
df = pd.read_csv('./insurance - insurance.csv')
# Numrical columns
num_cols = ['age', 'bmi', 'children', 'expenses']
# Create distribution plots
for col in num_cols:
    plt.figure(figsize=(8,5))
    sns.histplot(df[col], kde=True, bins=30, color='skyblue')
    plt.title(f'Distribution of {col.capitalize()}', fontsize=14)
    plt.xlabel(col.capitalize(), fontsize=12)
    plt.ylabel('Frequency', fontsize=12)
    plt.show()

```

Output:





Q6. Create count plots for all categorical columns (sex,smoker,ragion):

Use `sns.countplot()` for each column

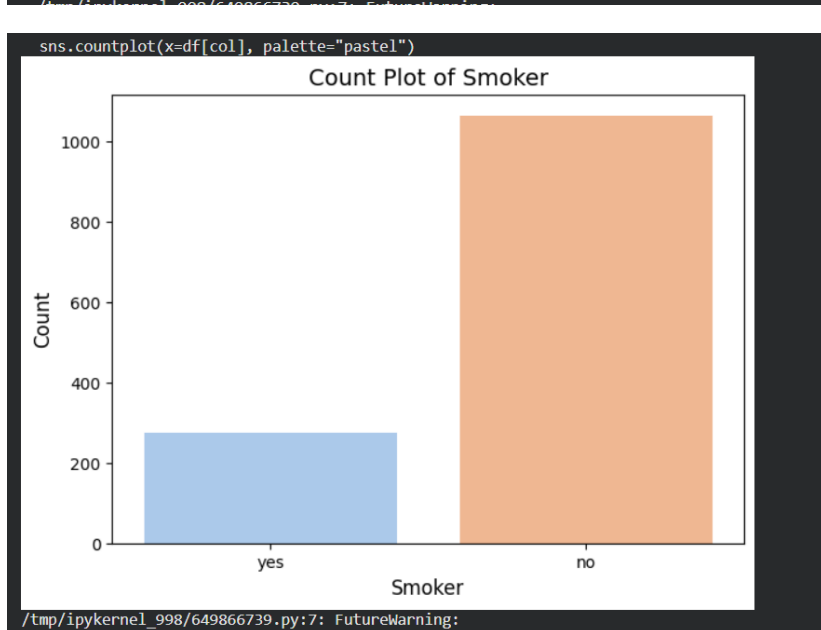
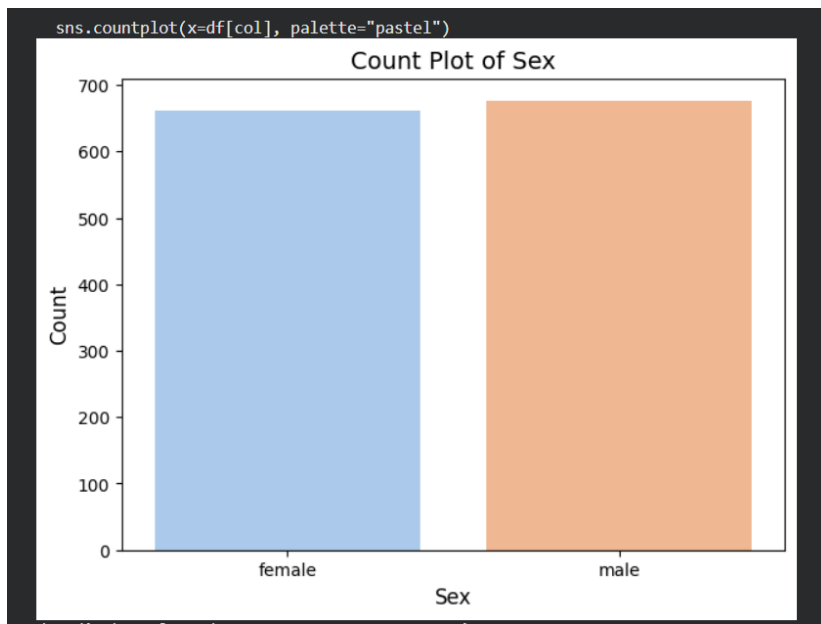
Add titles and labels for clarity

Code:

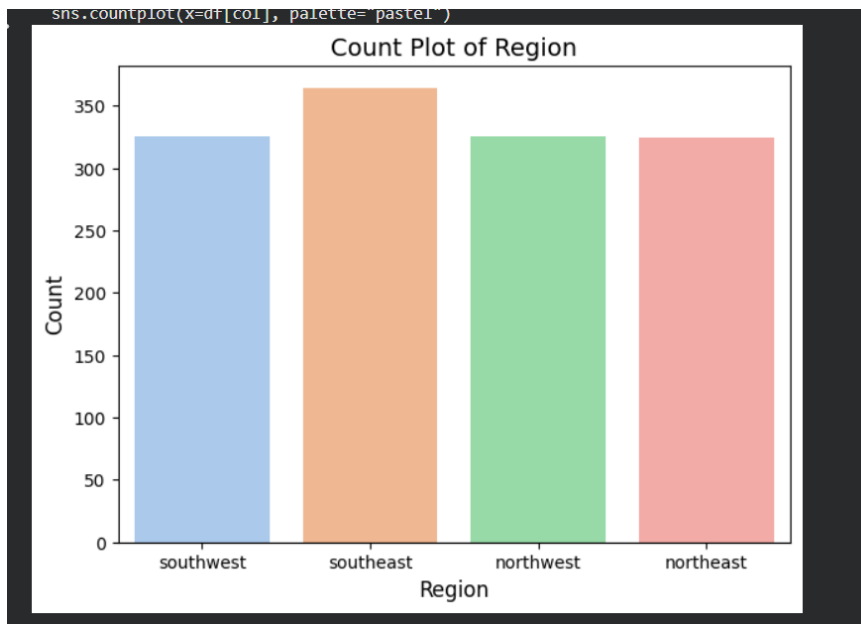
```
[22]
✓ 0s # Categorical columns
cat_cols = ['sex', 'smoker', 'region']

# Create count plots
for col in cat_cols:
    plt.figure(figsize=(7,5))
    sns.countplot(x=df[col], palette="pastel")
    plt.title(f'Count Plot of {col.capitalize()}', fontsize=14)
    plt.xlabel(col.capitalize(), fontsize=12)
    plt.ylabel('count', fontsize=12)
    plt.show()
```

Output:



/tmp/ipykernel_998/649866739.py:7: FutureWarning:



Q7. Create a heatmap to show the correlation between numerical variables. Use `sns.heatmap()`

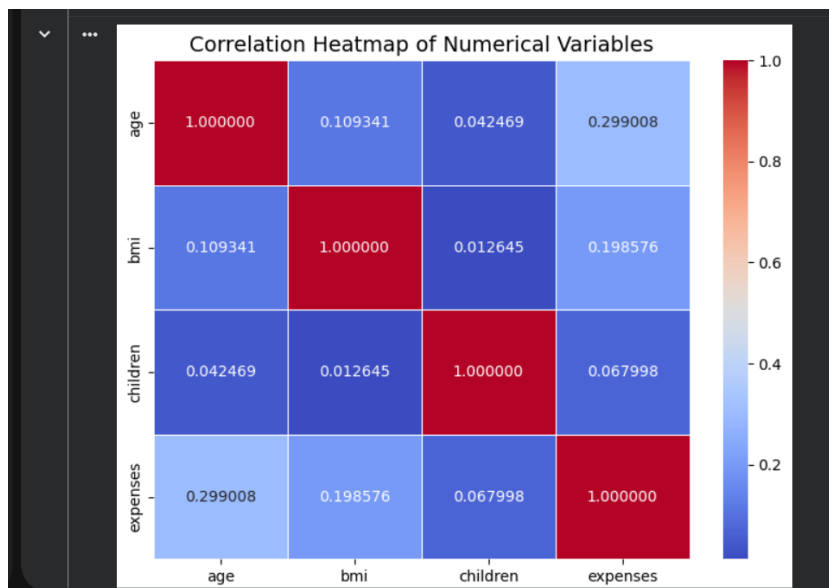
Annotate the values and use a suitable color palette

Code:

```
[8]
✓ 0s
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("insurance - insurance.csv")
# Select numerical columns
num_cols = ['age', 'bmi', 'children', 'expenses']
# Compute correlation matrix
corr = df[num_cols].corr()
# Plot heatmap
plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt="2f", linewidths=0.5)
plt.title("Correlation Heatmap of Numerical Variables", fontsize=14)
plt.show()
```

Output:



Q8. Perform the following analysis on the charges column:

Fine the average insurance charges

Fine the maximum and minimum charges

Group by smoker and compare the average charges for smokers vs non smokers

Code:

```
[10]
✓ 0s # 1. Average insurance charges
avg_charges = df['expenses'].mean()
print("Average Insurance Charges:", avg_charges)
# 2. Maximum and Minimum charges
max_charges = df['expenses'].max()
min_charges = df['expenses'].min()
print("Maximum Insurance Charges:", max_charges)
print("Minimum Insurance Charges:", min_charges)
# 3. Group by smoker and compare average charges
smoker_avg = df.groupby('smoker')['expenses'].mean()
print("\nAverage Charges by smoker Status:")
print(smoker_avg)
```

Output:

```
... Average Insurance Charges: 13270.422414050823
Maximum Insurance Charges: 63770.43
Minimum Insurance Charges: 1121.87

Average Charges by smoker Status:
smoker
no      8434.268449
yes     32050.231971
Name: expenses, dtype: float64
```


Q9. Create a boxplot to show the distribution of charges with respect to smoker and sex.

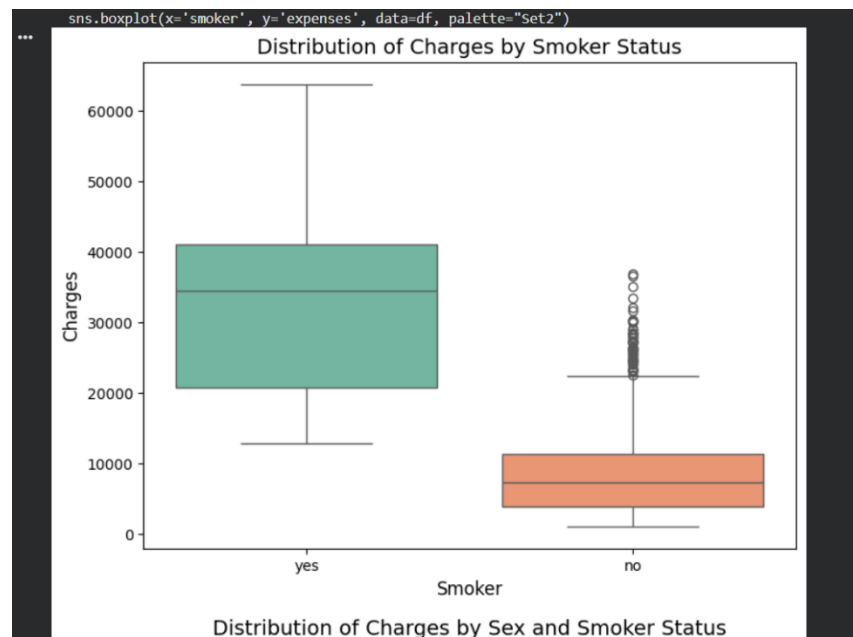
(You can create two separate boxplots or use hue parameter)

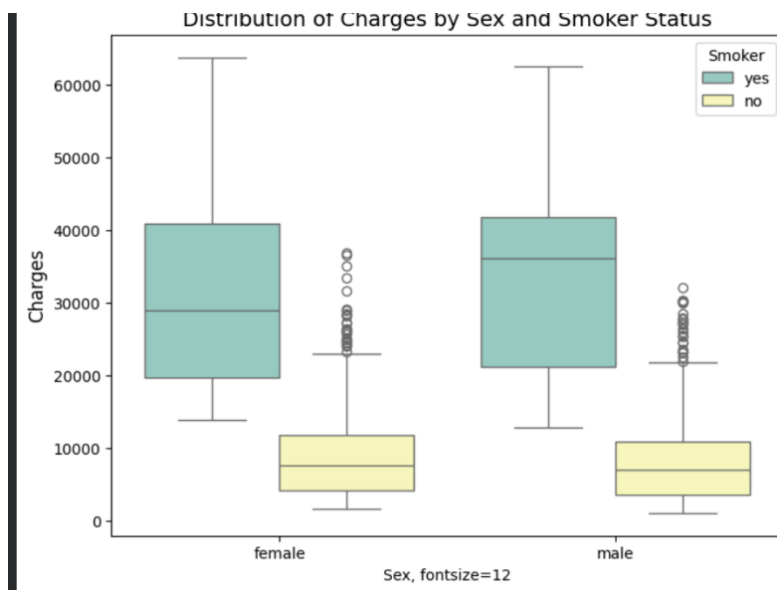
Code:

```
[12] ✓ 0s # Boxplot: Charges vs Smoker
plt.figure(figsize=(8,6))
sns.boxplot(x='smoker', y='expenses', data=df, palette="Set2")
plt.title("Distribution of Charges by Smoker Status", fontsize=14)
plt.xlabel("Smoker", fontsize=12)
plt.ylabel("Charges", fontsize=12)
plt.show()

# Boxplot: Charges vs Sex with Smoker as hue
plt.figure(figsize=(8,6))
sns.boxplot(x='sex', y="expenses", hue="smoker", data=df, palette="Set3")
plt.title("Distribution of Charges by Sex and Smoker Status", fontsize=14)
plt.xlabel("Sex", fontsize=12)
plt.ylabel("Charges", fontsize=12)
plt.legend(title="Smoker")
plt.show()
```

Output:





Q10. (Mini Project / Analysis Summary)

Write a short summary (in comments or markdown) answering the following:

1. What is the average age and BMI of the customers?
2. How does smoking affect insurance charges?
3. Which region has the highest number of customers?
4. Any interesting observations from the visualizations?

Insurance Dataset Analysis Summary

1. **Average Age & BMI**
 - The average age of customers is around **39 years**.
 - The average BMI is approximately **30**, which is slightly above the healthy range.
2. **Impact of Smoking on Charges**
 - Smoking has a **huge impact** on insurance charges.
 - Average charges for **smokers** are significantly higher (~ 32k-34k) compare to **non-smokers** (~ 8k-9k).
3. **Region with Highest Customers**
 - The **southeast region** has the highest number of customers, followed by southwest and northwest.
4. **Interesting Observations from Visualizations**
 - **Distribution plots** show most customers are young adults (18-30) with BMI clustering around 25-35.
 - **Boxplots** reveal smokers consistently have higher charges, regardless of sex.

- ****Heatmap**** shows a moderate positive correlation between ****age and expenses****, while BMI has weaker correlation.
- ****Count plots**** highlight that the dataset is fairly balanced by sex, but smoker counts are much lower than non-smokers.

Overall, the dataset clearly demonstrates that **smoking is the strongest factor driving higher insurance costs****, while age and BMI also contribute moderately.**

OPTIONAL TASK

Q1.

```
[1] import numpy as np
import pandas as pd

[5] df = pd.read_csv("/content/Chocolate_Sales.csv")
df.head()
```

	Order_ID	Product	Country	Channel	Salesperson	Order_Date	Discount_Pct	Price_per_Box	Marketing_Spend	Boxes_Shipped	Amount
0	ORD-069833	Truffle Gift Box	Australia	Retail	Arjun Mehta	2022-12-11	3.5	13.72	202.03	71	912.31
1	ORD-090726	85% Dark Bar	Australia	Retail	Arjun Mehta	2023-03-14	9.4	3.30	55.18	84	245.91
2	ORD-042159	70% Dark Bar	Japan	Retail	Hannah Müller	2023-12-21	4.9	18.21	60.65	35	583.7
3	ORD-197166	Hazelnut Milk Bar	Germany	Retail	Arjun Mehta	2023-12-18	15.0	2.66	52.00	92	211.27
4	ORD-112162	Almond Crunch Bar	Australia	Retail	Yuki Sato	2023-08-18	4.4	2.75	187.44	214	549.69

Q2.

```
[6] # Dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Order_ID              200000 non-null object  
 1   Product               200000 non-null object  
 2   Country               200000 non-null object  
 3   Channel               200000 non-null object  
 4   Salesperson           200000 non-null object  
 5   Order_Date            199563 non-null object  
 6   Discount_Pct          199511 non-null float64 
 7   Price_per_Box         199543 non-null float64 
 8   Marketing_Spend       199539 non-null float64 
 9   Boxes_Shipped         200000 non-null int64  
10   Amount                200000 non-null object  
dtypes: float64(3), int64(1), object(7)
memory usage: 16.8+ MB
```

Q3.

```
[7] # Statistical summary
df.describe()
```

	Discount_Pct	Price_per_Box	Marketing_Spend	Boxes_Shipped
count	199511.000000	199543.000000	199539.000000	200000.000000
mean	13.107457	5.911507	95.450141	139.584105
std	6.374477	4.954976	66.243760	118.665393
min	0.000000	2.040000	4.300000	-1642.000000
25%	8.300000	2.950000	52.270000	67.000000
50%	12.600000	3.280000	78.850000	113.000000
75%	17.500000	4.570000	118.710000	180.000000
max	38.300000	21.650000	798.222443	3811.000000

Q4.

```
[8] ✓ 0s # Missing values
df.isnull().sum()
```

...	0
Order_ID	0
Product	0
Country	0
Channel	0
Salesperson	0
Order_Date	437
Discount_Pct	489
Price_per_Box	457
Marketing_Spend	461
Boxes_Shipped	0
Amount	0

dtype: int64

Q5.

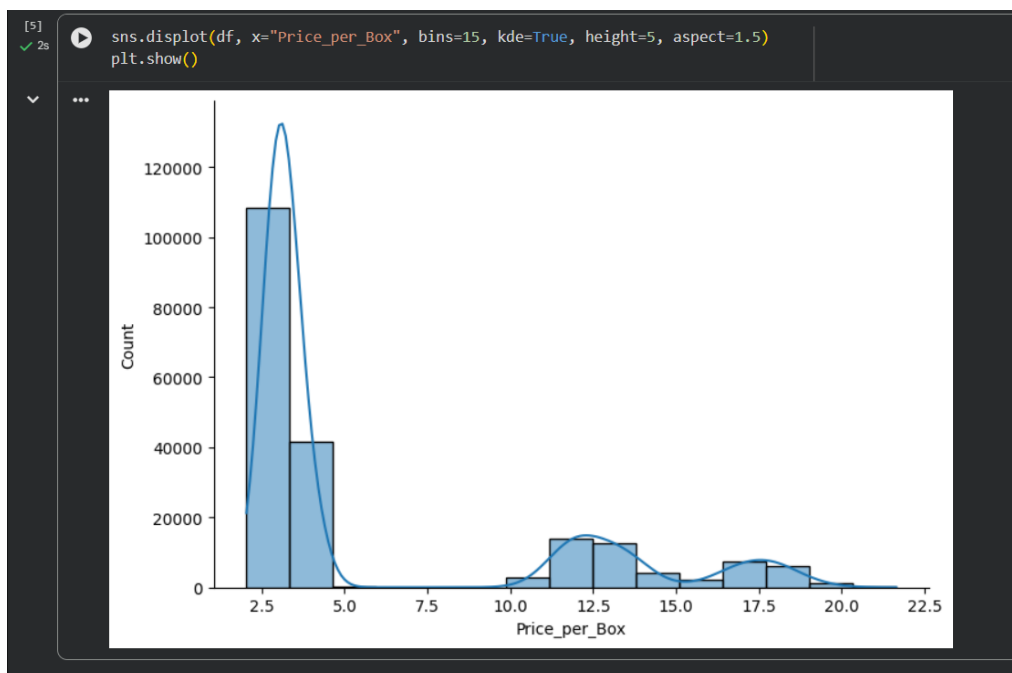
```
[9] ✓ 0s # Duplicate values
df.duplicated().sum()
```

```
... np.int64(0)
```

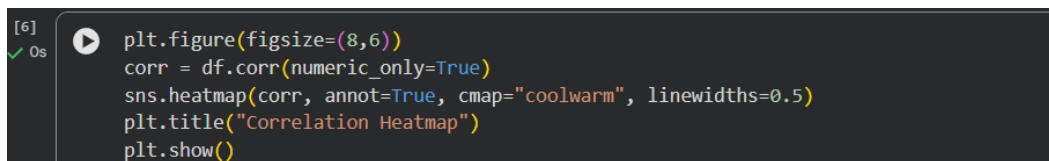
Q6.

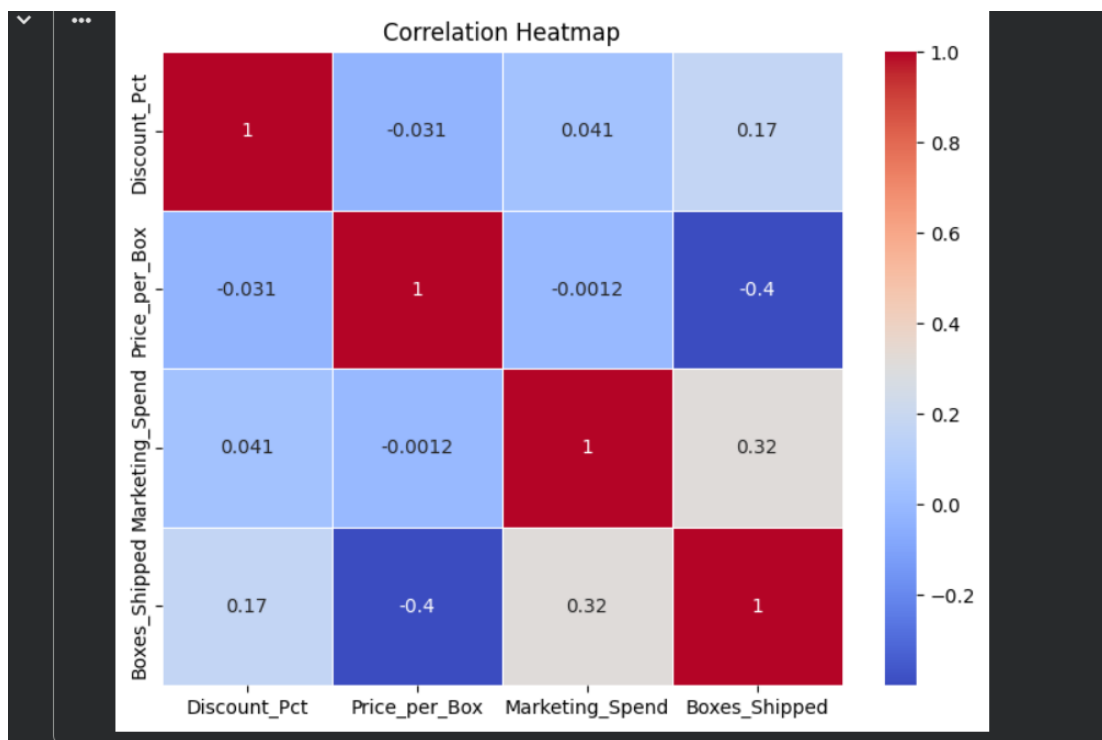


Q7.



Q8.

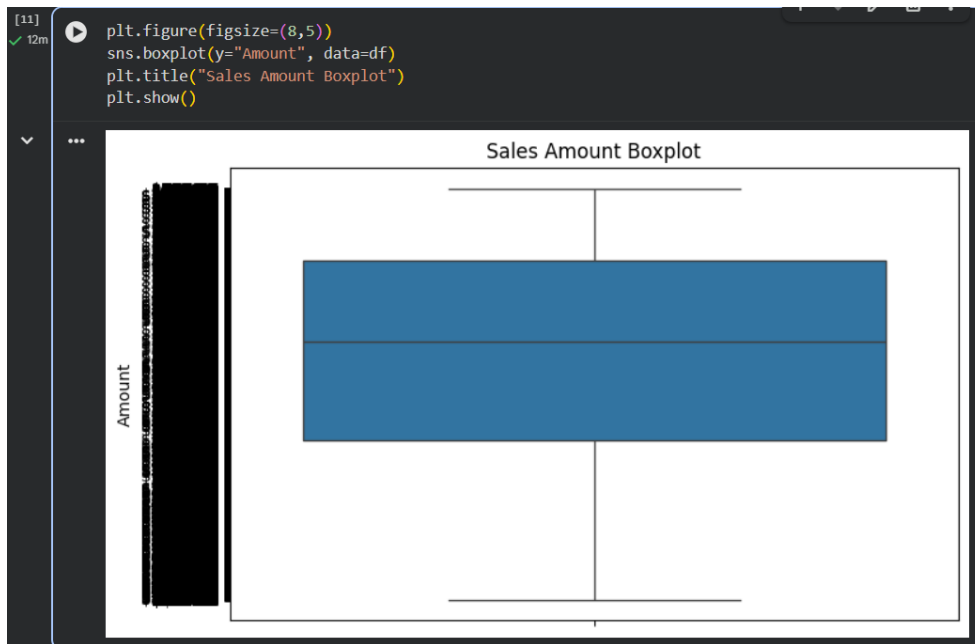




Q9.



Q10.



Summary:

The Chocolate Sales dataset was analyzed using Python, Pandas, Matplotlib, and Seaborn. The dataset was explored by checking its structure, missing values, and summary statistics. Visualization such as histogram, distribution plot, and correlation heatmap were used to understand the distribution of sales and relationships between numerical variables.

The analysis showed that sales amounts vary across transaction, and the histogram helped identify the overall distribution of sales values. The heatmap indicated the strength of correlation between numerical columns, helping to understand how different variables are related. Overall, the dataset provided useful insights into chocolate sales patterns and can help in making better business decision based on sales performance.